SWE-EVO

Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios

Le et al. (2026) | FPT Software AI Center · University of Melbourne · VinUniversity

Paper Review Presentation | AI Agents Systems Course

48	7	21	874	25%
Tasks	Repos	Avg Files	Avg Tests	Best Score

A — Prior Literature Review

From HumanEval to SWE-Bench: evolution of coding benchmarks

B — Research Questions

Main and secondary questions addressed by SWE-EVO

C — Evaluation Metrics

Resolved Rate, Fix Rate, Apply Rate and their adequacy

D — Main Results

Capability gap, failure modes, difficulty scaling

E — Limitations & Critical Analysis

What the authors acknowledge and what they miss

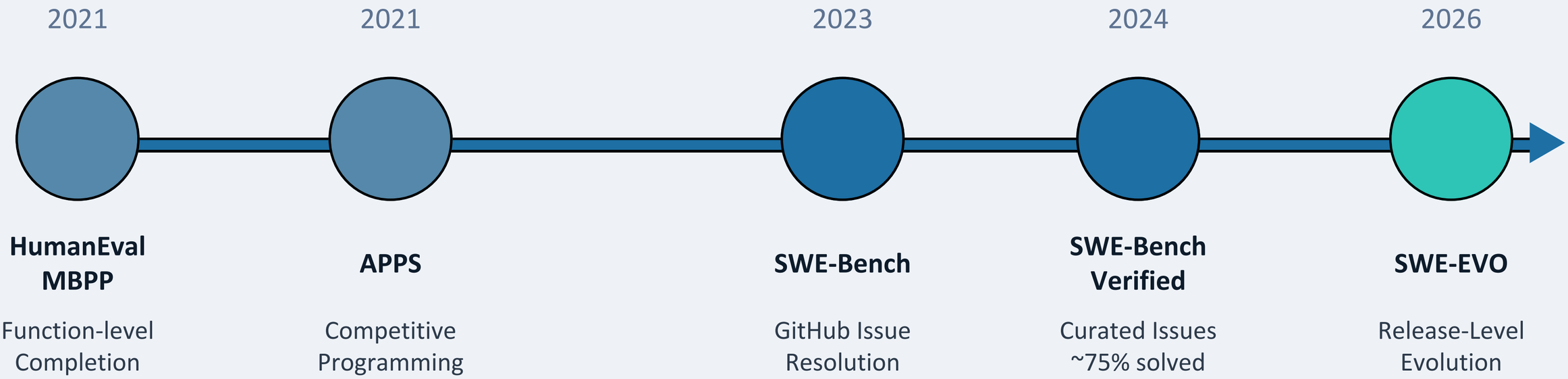
F — Future Directions

Extensions and novel research directions

Prior Literature Review

How the field evolved to SWE-EVO

From Functions to Releases: Benchmark Evolution



Era 1: Function-Level

Single-file tasks, increasingly saturated

Era 2: Repository-Level

Real GitHub issues, one PR per task

Era 3: Software Evolution

Multi-PR, full release cycles

SWE-Bench nears saturation at ~75% on Verified - but its isolated issue framing misses the core SE challenge: continuous codebase evolution

The Rise of AI Coding Agents

Key Agent Systems

SWE-agent (2024)

Agent-computer **interface** enabling automated software engineering

OpenHands (2024)

Open multi-agent platform evaluated on 15 benchmarks

AutoCodeRover (2024)

Pairs LLMs with **AST**-based search for precise **code localization**

Agentless (2024)

Simple localization-repair pipeline, competitively effective

Devin AI (2024)

Commercial autonomous software engineer (Cognition)

Industry Adoption (DORA 2025)

DORA = DevOps Research and Assessment

>90%

of engineering teams now use generative AI in SE workflows

Up from 61% in 2024

Three parallel trends driving the field: multi-agent coordination, smarter tool use, and training on real SE data

B

Research Questions

What problem does this paper address, and why does it matter?

The Motivation Gap

Existing benchmarks for AI coding agents focus on **isolated, single-issue tasks** such as fixing a bug or adding a small feature. However, **real-world software engineering is a long-horizon endeavor**:

- developers **interpret high-level requirements**
- **coordinate changes across many files**
- and **evolve codebases over multiple iterations while preserving functionality**.

The Motivation Gap

SWE-EVO, a benchmark for this long-horizon software evolution challenge. Constructed from:

- Release transitions
- in seven mature open-source Python projects
- Comprises 48 release-sized tasks
- Each task requires **multi-step modifications** spanning an average of 21 files **validated** against test suites averaging 874 tests per instance.

The Motivation Gap: What SWE-Bench Misses

80%

of SE effort: maintaining & evolving code

	SWE-Bench	SWE-EVO
Task Input	1 GitHub Issue	Release Notes (many PRs)
Output	1 Patch File	Multi-file Patch
Avg Scope	~2 files, ~33 lines	~21 files, ~610 lines
FAIL→PASS Tests	~9 per task	~81 per task
PASS→PASS Tests	few	~793 per task
PRs / Task	1 PR	Multiple PRs

Real SE = continuous evolution: multi-feature releases, coordinated cross-file changes, regression preservation. SWE-EVO captures this. SWE-Bench does not.

Research Questions

MAIN RESEARCH QUESTION

This gap between benchmark tasks and real-world evolution scenarios motivates our central research question: *Given an existing codebase, can multi-agent LLM systems autonomously evolve the system in response to dynamic input requirements, demonstrating sustained planning, adaptability, and innovation across long-horizon tasks?*

Secondary Research Questions

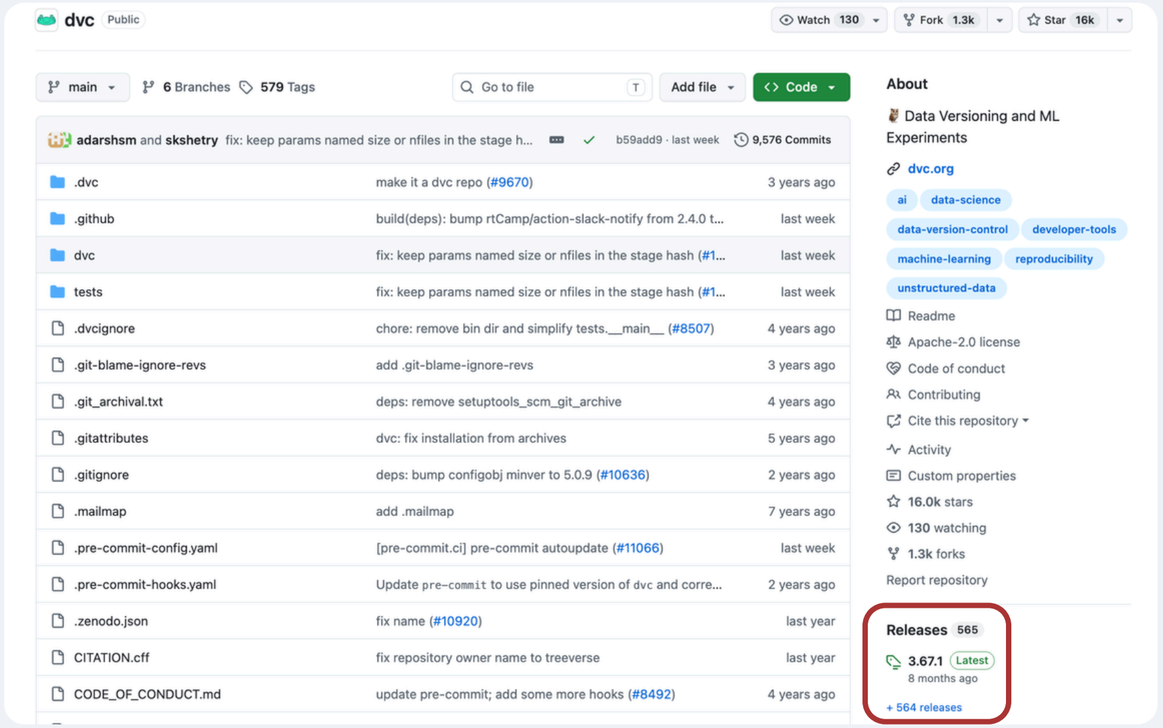
RQ1 How large is the **performance gap** between **SWE-EVO** and **SWE-Bench**? Do high SWE-Bench scores transfer to evolution tasks?

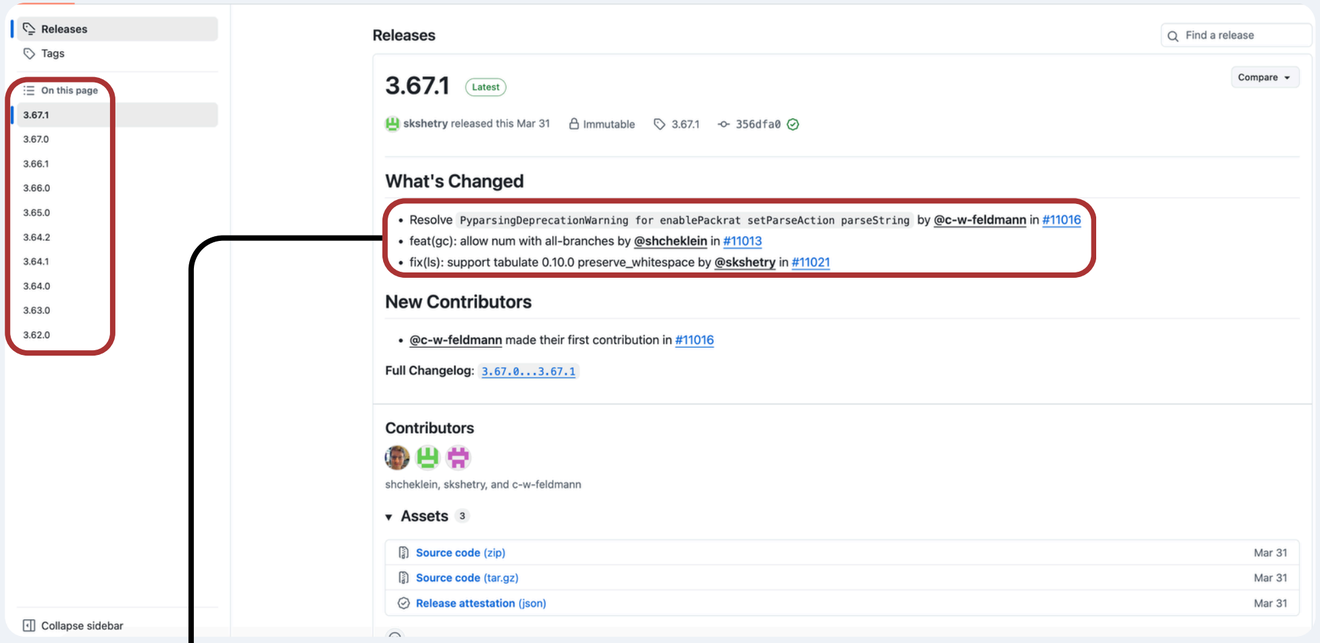
RQ2 What **failure modes** do agents exhibit? Do they differ systematically across model families and scaffolds?

RQ3 How does task difficulty scale with **PR count**?

RQ4 Does providing **additional context** (linked PR/issue text) meaningfully improve agent performance?

Benchmark Construction







Benchmark Construction

CONSTRUCTION PIPELINE

STAGE I
Repository Selection
Inherited from SWE-bench and SWE-gym. No new repos chosen from scratch.
Seed pool

›

STAGE II
Candidate Filtering
Keep only instances whose base commit matches a release version tag.
Release snapshots

›

STAGE III
Execution Filtering
Run tests before and after patch. Keep only instances with at least one FAIL to PASS test.
Verifiable delta

›

FINAL
Human Curation
Manual check of alignment between release note, PR context, gold patch, and tests.
48 instances

FUNNEL: HOW MANY INSTANCES AT EACH STAGE

SWE-bench + SWE-gym
Starting seed pool of task instances

~hundreds
After automatic filtering (exact number not reported)

48
After human curation — final benchmark

THE 7 REPOSITORIES INCLUDED

dvc (26 instances) · dask (8) · requests (4) · pydantic (3) · modin (3) · conan (2) · scikit-learn (2)

Benchmark Construction

		Mean	Max
Issue Text	Length (Words)	2390.5	22344
Codebase	# Files (non-test)	363	1046
	# Lines (non-test)	78K	272K
Gold Patch	# Lines edited	610.5	4113
	# Files edited	20.9	105
	# Func. edited	51.0	379
Tests	# Fail to Pass	81.4	2774
	# Total	874.0	8552

Table 1 | Average and maximum numbers characterizing different attributes of a SWE-EVO task instance. Statistics are micro-averages calculated without grouping by repository.

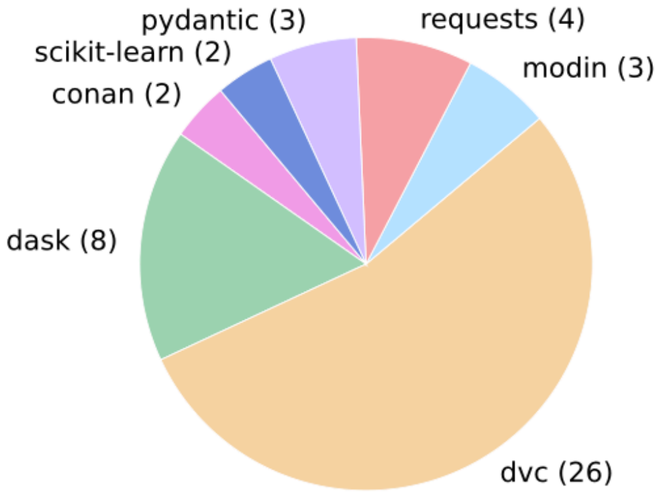


Figure 3 | Distribution of SWE-EVO tasks (in parenthesis) across 7 open source GitHub repositories that each contains the source code for a popular, widely downloaded PyPI package.

The Motivation Gap: What SWE-Bench Misses

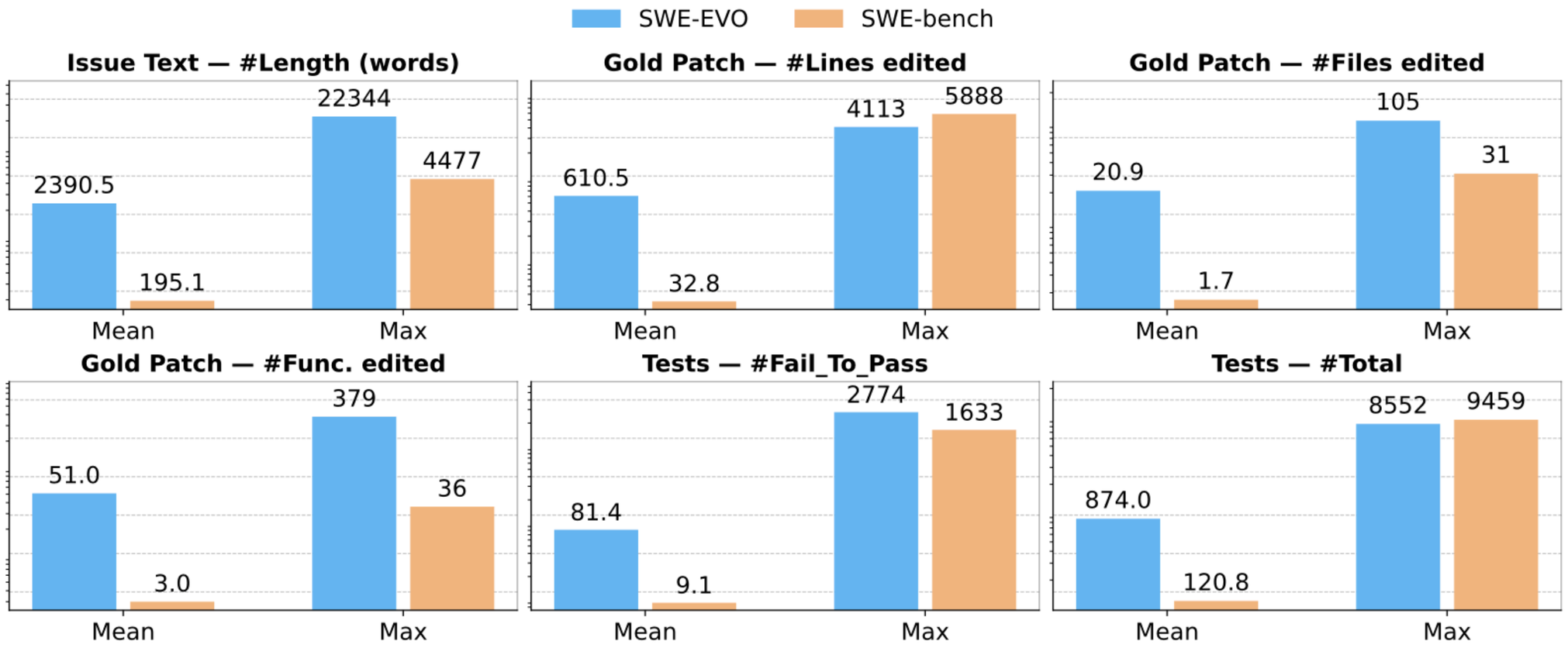


Figure 4 | SWE-EVO is markedly more demanding than SWE-Bench, with longer specifications, broader patch scope, and heavier test suites in both mean and max.

C

Evaluation Metrics

How do we measure success on long-horizon evolution?

Evaluation Metrics

Resolved Rate

Primary (Hard Score)

Binary: 1 only if ALL FAIL_TO_PASS and PASS_TO_PASS tests succeed

```
Score = 1  iff  all tests pass
Score = 0  otherwise
```

- Clear, comparable to SWE-Bench
- Hides partial progress

Fix Rate

New Soft Score (novel!)

Fraction of FAIL→PASS tests fixed. Set to 0 if any PASS→PASS test fails

```
Fix = |{t in F : passes}| / |F|
(= 0 if any PASS_TO_PASS fails)
```

- Captures meaningful partial progress
- Weights all tests equally

Fix Rate = 1 iff Resolved Rate = 1

Patch Apply Rate

Auxiliary Diagnostic

% of generated patches syntactically valid and applicable without errors

```
Applied patches / Total patches
```

- Diagnoses basic agent capability
- No semantic correctness measure

Are the Metrics Adequate? A Critical Assessment

Strengths

- Fix Rate prevents 'all-or-nothing' unfairness on tasks with 800+ tests (where failing a single test represents only a tiny fraction of the work)
- Regression constraint (PASS→PASS) aligns with real SE quality standards
- Execution-based: objective, deterministic, fully reproducible
- Consistent with SWE-Bench: enables cross-benchmark comparison

Concerns

- Tests weighted equally: fixing a trivial edge case = fixing a critical feature
- No code quality, maintainability, or readability measure
- One PASS→PASS failure zeroes Fix Rate regardless of severity - very strict
- No semantic evaluation: ugly but correct patch scores the same as clean code

Future Directions: Weighted Fix Rate (by test importance) · Semantic correctness via LLM-as-judge · Code complexity / style metrics · Patch minimality score

D

Main Results

A striking capability gap and what causes it

Main Results

	Model	SWE-EVO (OpenHands)		SWE-EVO (SWE-agent)		SWE-Bench Verified (bash only)
		% Resolved	% Apply	% Resolved	% Apply	% Resolved †
OpenAI	gpt-5.4	**25.00**	97.92	**25.00**	97.92	—
	gpt-5.2	18.75	100	22.92	97.92	72.80
	gpt-5-08-07	18.75	100	20.83	100	65.00
	gpt-5-mini-08-07	10.42	97.92	10.42	100	59.80
	gpt-5-nano-08-07	4.17	85.42	4.17	100	34.80
	o3-2025-04-16	4.17	93.75	6.25	100	58.40
	gpt-4.1-2025-04-14	2.08	87.50	10.42	97.92	39.58
	gpt-4o-2024-11-20	6.25	97.92	6.25	100	21.62
	gpt-oss-120b	2.08	100	6.25	100	26.00
DeepSeek	deepseek-v3p2	20.83	95.83	23.40	95.83	70.00
	deepseek-v3p1	16.67	97.92	10.42	100	—
	deepseek-r1-0528	10.42	100	8.33	100	57.60
Zhipu AI	glm-5	8.33	97.92	37.50	100	72.80
	glm-4p7	4.17	100	39.58	97.92	—
	glm-4p5	16.67	97.92	16.67	100	54.20
Qwen	qwen3-coder-480b-a35b	14.58	97.92	14.58	97.92	55.40
Moonshot AI	kimi-k2p5	22.92	97.92	25.00	97.92	70.80
	kimi-k2-instruct	16.67	100	18.75	100	43.80

Table 2 | Results on SWE-EVO with *release note + PR/issue context*: Resolved and Apply rates for OpenHands and SWE-agent. †Results reported on swebench.com (Jimenez et al., 2024). "—" indicates that the result is not reported on the SWE-Bench website.

Main Results

INSIGHT 1

No model exceeds 25% — benchmark has genuine headroom

Best-performing models on SWE-EVO (best scaffold)

MODEL	PROVIDER	BEST SCORE	STATUS
gpt-5.4	OpenAI	25.00%	Ceiling
kimi-k2p5	Moonshot AI	25.00%	Ceiling
deepseek-v3p2	DeepSeek	23.40%	Near top
gpt-5.2	OpenAI	22.92%	Near top
glm-4p7	Zhipu AI	39.58%	Outlier ⚠

glm-4p7 at 39.58% is SWE-agent only — see Insight 3 (scaffold sensitivity)

Main Results

INSIGHT 2

SWE-Bench scores don't transfer to SWE-EVO

Models that score high on SWE-Bench still collapse on SWE-EVO

MODEL	SWE-BENCH VERIFIED	SWE-EVO (BEST)	DROP
o3	58.40%	6.25%	−52.2 pp
gpt-5.2	72.80%	22.92%	−49.9 pp
deepseek-v3p2	70.00%	23.40%	−46.6 pp
kimi-k2p5	70.80%	25.00%	−45.8 pp
gpt-4o	21.62%	6.25%	−15.4 pp

Every model drops significantly — ranking on SWE-Bench does not predict ranking on SWE-EVO

Main Results

INSIGHT 3

Scaffold sensitivity is real — especially for glm models

Same model, different scaffold → dramatically different results

MODEL	OPENHANDS	SWE-AGENT	GAP
glm-4p7	4.17%	39.58%	+35.4 pp
glm-5	8.33%	37.50%	+29.2 pp
gpt-4.1	2.08%	10.42%	+8.3 pp
gpt-5.4	25.00%	25.00%	0 pp
deepseek-v3p1	16.67%	10.42%	−6.3 pp

Conclusions about "model capability" must always specify the scaffold used

Main Results

INSIGHT 4

Scaling holds within model families

Larger variants consistently outperform smaller ones

FAMILY	MODEL	BEST SCORE
OpenAI	gpt-5.4	25.00%
	gpt-5.2	22.92%
	gpt-5-08-07	20.83%
	gpt-5-mini	10.42%
	gpt-5-nano	4.17%
DeepSeek	deepseek-v3p2	23.40%
	deepseek-v3p1	16.67%
	deepseek-r1	10.42%

Scaling law holds within families — but different families are not directly comparable

Main Results

INSIGHT 5

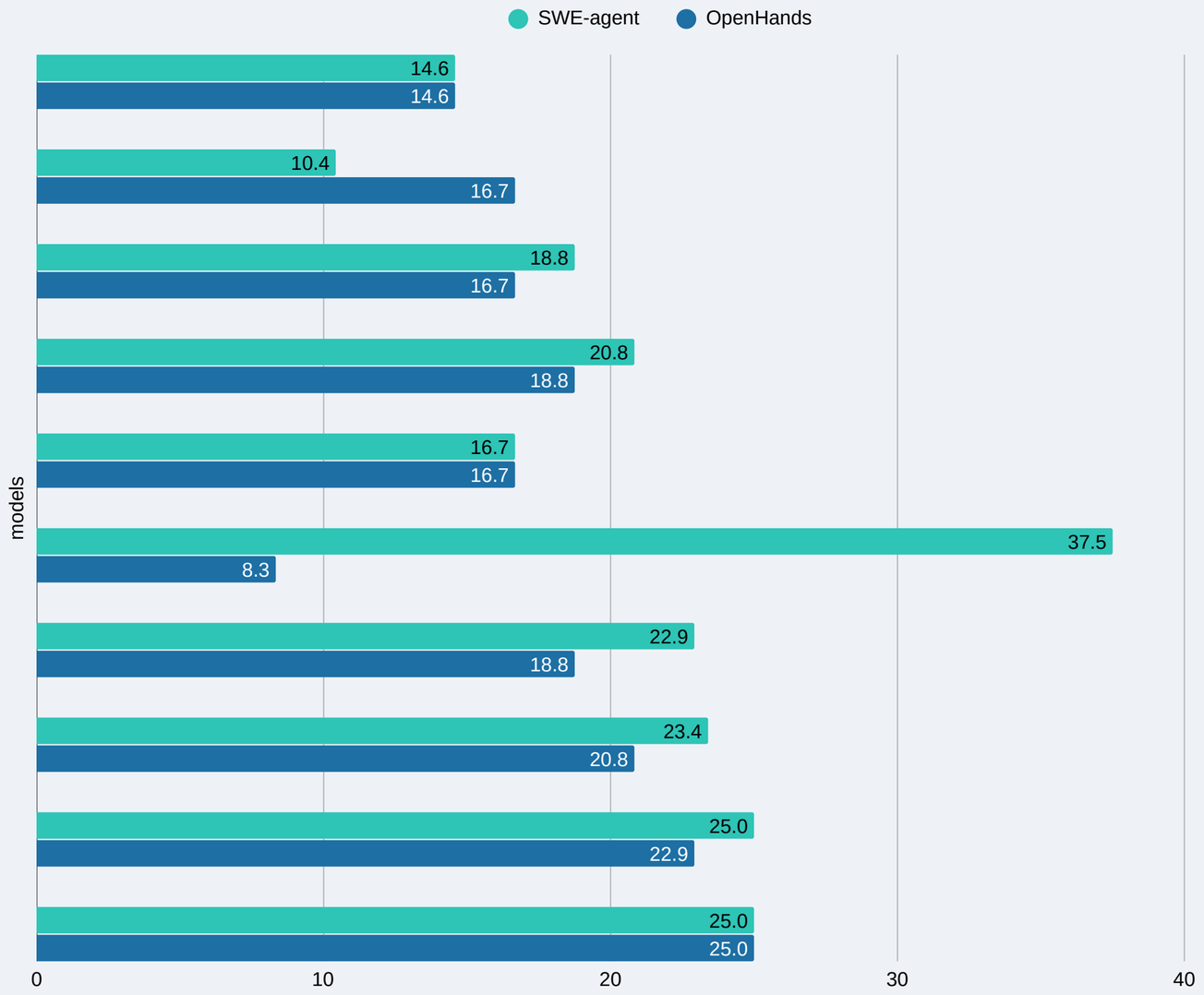
Semantics, not syntax, is the barrier

Patch Apply Rate ~98% across all models — agents can write valid code, but not correct code

MODEL	APPLY RATE	RESOLVED RATE	GAP
gpt-5.2	100%	18.75%	−81.3 pp
deepseek-v3p2	95.83%	20.83%	−75.0 pp
gpt-5-nano	100%	4.17%	−95.8 pp
o3	93.75%	4.17%	−89.6 pp
Average	~97%	~12%	~85 pp

The bottleneck is understanding requirements and coordinating changes — not generating syntactically valid patches

Main Results



Key Findings

No model exceeds 25% - benchmark has genuine headroom

glm-5*: 8% (OpenHands) vs 37% (SWE-agent) - **scaffold sensitivity is real**

Scaling holds: larger models outperform smaller ones within families

Patch Apply Rate ~98% across all - **semantics, not syntax, is the barrier**

Fine-Grained Analysis with Fix Rate

Both **gpt-4.1** (stronger, more expensive) and **gpt-oss-120b** (open weights) score 2.08% Resolved Rate.

Fix Rate reveals the difference: gpt-4.1 **fixes more failing tests per task** (4.65% vs 2.08%), but never enough to fully resolve any instance. Binary scoring makes them look equal. Fix Rate does not.

Fix Rate shows that gpt-4.1 is actually making meaningful progress that the binary metric completely hides.

Trajectory Failure Modes Analysis - Why Do Agents Fail?

An LLM-as-a-judge analysis of unresolved SWE-agent trajectories.The judge assigns one primary label from 6 label options (the labels are qualitative diagnostics rather than execution-based scores). Human agreement validation remains future work.

The taxonomy separates **low-level execution failures** from **semantic failures**:

- **Syntax Error** and **Tool-Use** capture **invalid patches** or **failed interaction with the agent tools**.
- **Incorrect Implementation** and **Instruction Following** capture **wrong logic** or **misread requirements**
- **Stuck in Loop** and **Gave Up Prematurely** capture **unproductive** or **prematurely terminated trajectories**
- Other covers rare ambiguous cases.

Trajectory Failure Modes Analysis - Why Do Agents Fail?

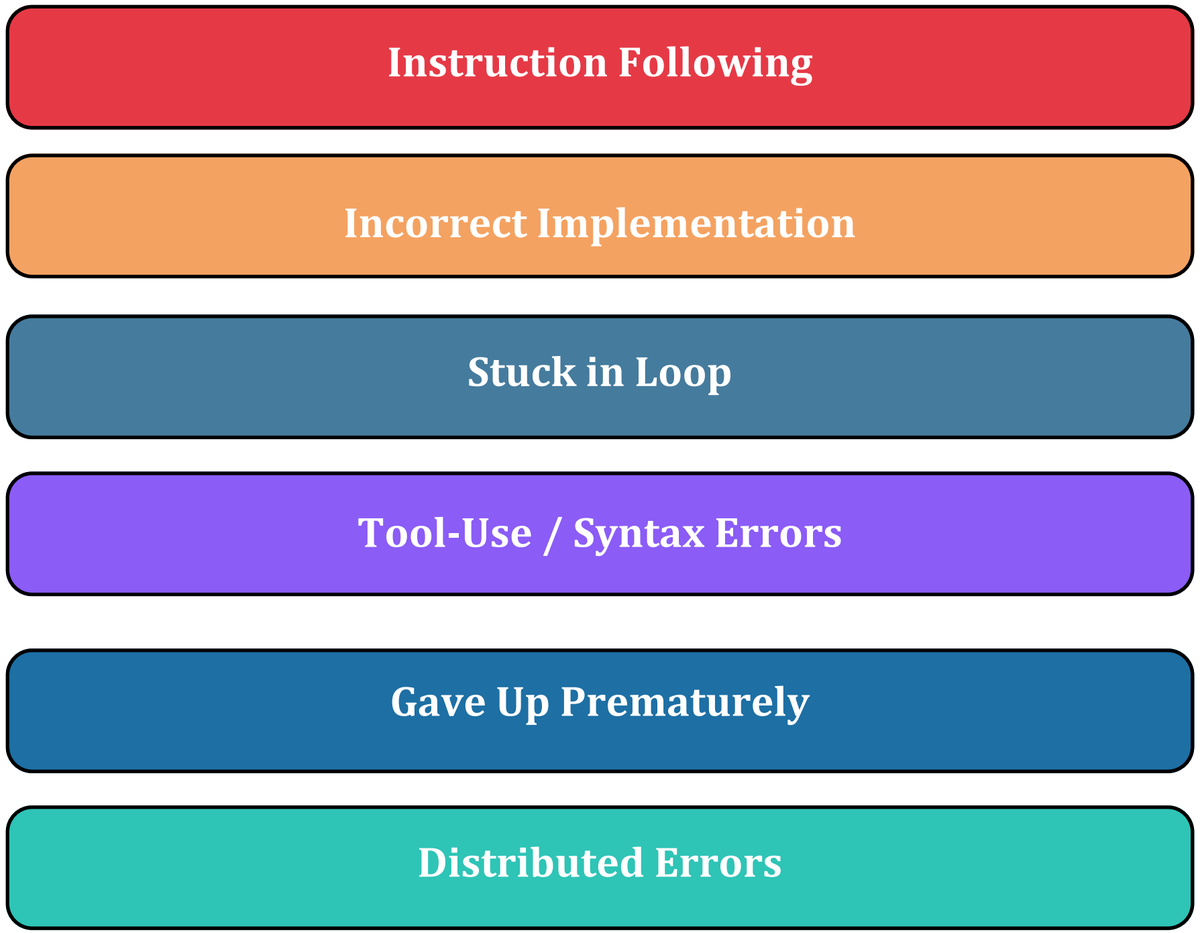
Key pattern: failure type shifts with model capability:

GPT family:

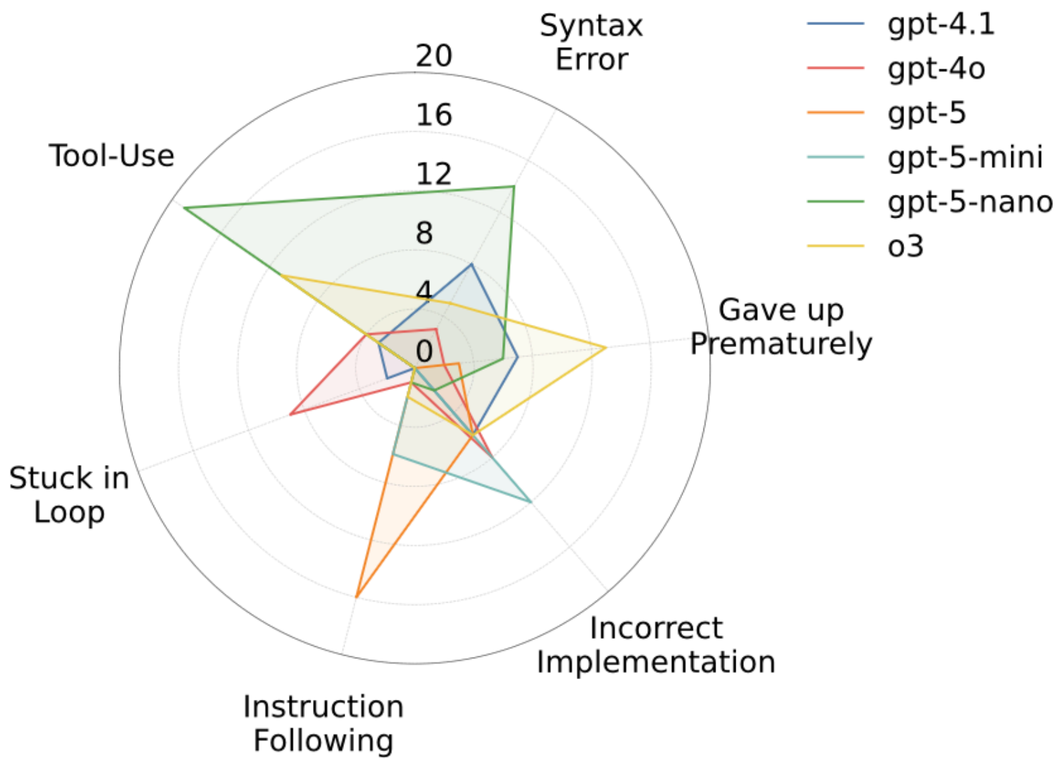
- Strongest models (gpt-5)
- Smaller variants (gpt-5-mini, gpt-5-nano)
- older models (o3, gpt-4.1, gpt-4o)

Open & Hybrid Models:

- kimi-k2, qwen3, gpt-oss-120b → Incorrect Implementation
- deepseek-r1 → Stuck in Loop
- glm-4p5 → Distributed Errors



Failure Mode Analysis: Why Do Agents Fail?



(a) GPT-series models.

gpt-5
OpenAI · Largest, most capable

Instruction Following (>60%)

Strong enough to write code and use tools — the bottleneck is interpreting complex release notes. Commits to the wrong reading and runs with it.

"gpt-5 rarely fails due to syntax or tool issues; over 60% of failures come from Instruction Following"

gpt-5-mini / gpt-5-nano
OpenAI · Smaller, cheaper variants

Tool-Use · Syntax · Incorrect Impl.

Never reach the stage of misunderstanding requirements — fail on basic execution first. Wrong file paths, broken patches, malformed commands.

"Smaller variants show increasing Incorrect Implementation, Tool-Use, and Syntax Error failures"

o3
OpenAI · Reasoning model

Gave Up Prematurely (dominant)

Despite being a reasoning model, gives up early when facing multi-PR complexity. Declares failure rather than trying a different approach.

"older models (o3, gpt-4.1, gpt-4o) exhibit more looping and early-termination issues"

gpt-4.1 / gpt-4o
OpenAI · Previous generation

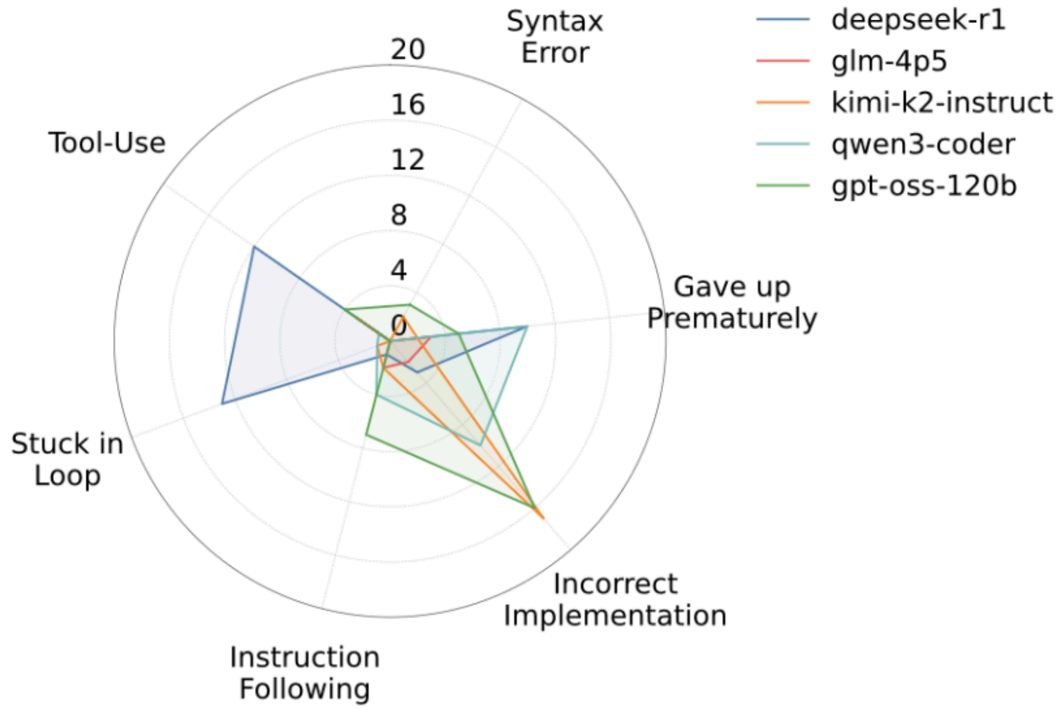
Stuck in Loop + Gave Up Prematurely

Not optimized for long-horizon agentic work. Either repeat unproductive actions in a loop, or stop entirely when they hit a wall.

"older models (o3, gpt-4.1, gpt-4o) exhibit more looping and early-termination issues"

Figure 5 | Failure mode distribution for SWE-agent with several model trajectories of unresolved instances. Each instance is automatically labeled using `gpt-5-mini` (OpenAI, 2025b) with categories from Table 8.

Failure Mode Analysis: Why Do Agents Fail?



(b) Open-source models.

deepseek-r1
DeepSeek (China) · Reasoning model

Stuck in Loop

Like o3, it's a reasoning model — same over-thinking pattern. Also frequently fails on execution, getting stuck before making real progress.

qwen3-coder
Alibaba (China) · Code-specialized

Incorrect Implementation

Specialized for coding tasks but struggles with the semantic complexity of multi-PR evolution. Locates changes correctly, implements them wrongly.

glm-4p5
Zhipu AI (China) · General model

Distributed Errors

No single dominant failure mode — errors spread evenly across all categories. Suggests no clear strength to fall back on when facing complex tasks.

kimi-k2-instruct
Moonshot AI (China) · Agentic model

Incorrect Implementation

Understands the task and uses tools correctly — but the logic it writes is wrong. Finds the right place in the code, just doesn't fix it correctly.

gpt-oss-120b
OpenAI · Open-weight, 120B params

Incorrect Implementation

Open-weight model by OpenAI — weights are public. Despite its size, semantic reasoning lags behind closed models of similar scale.

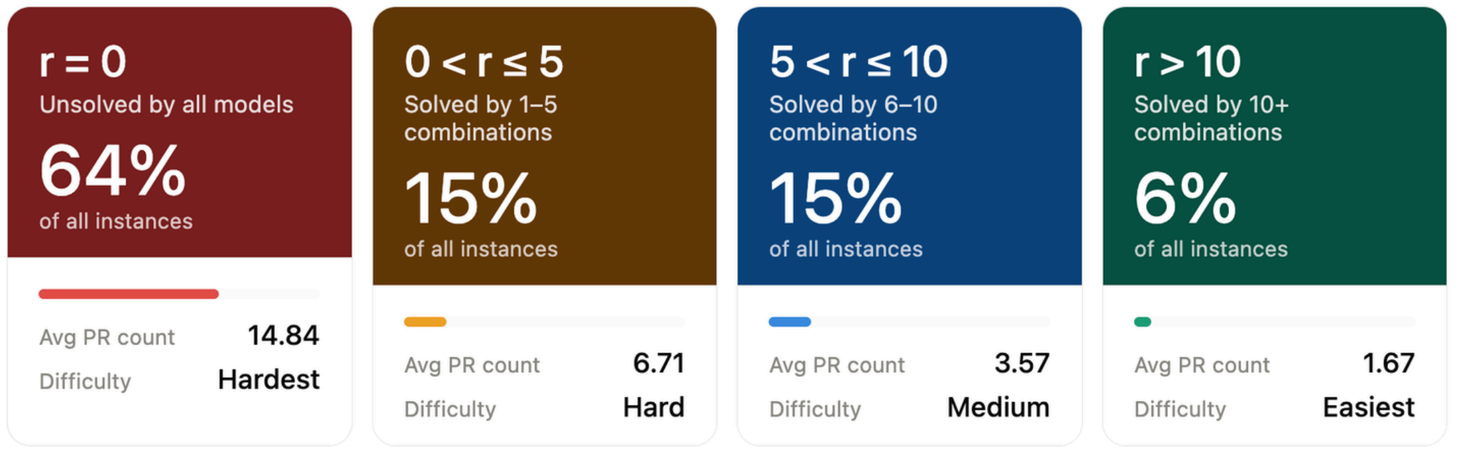
Figure 5 | Failure mode distribution for SWE-agent with several model trajectories of unresolved instances. Each instance is automatically labeled using `gpt-5-mini` (OpenAI, 2025b) with categories from Table 8.

Difficulty Analysis: What Makes Tasks Hard?

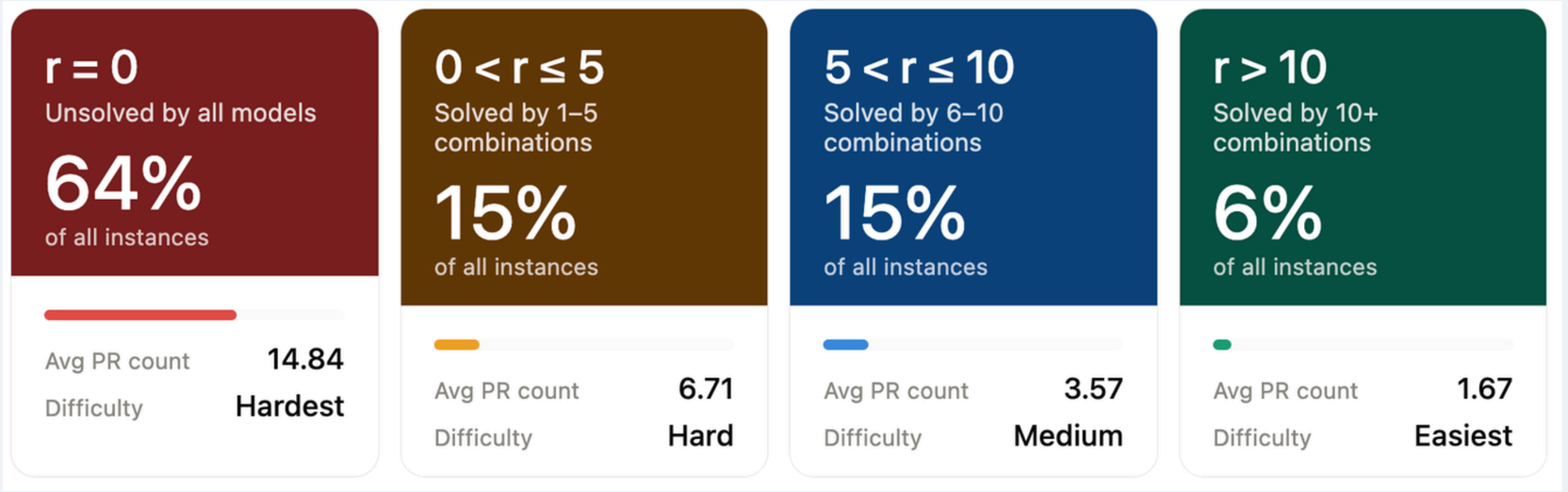
How can we define instance difficulty in this benchmark?

Benchmark Instances difficulty varies widely across multiple axes: PR count, specification length, files and lines edited, functions edited, test-suite size.

Rather then trying to group by those attributes to difficulties, they grouped the instances by the number of models that were able to solve them, and then characterize them by those attributes.



Difficulty Analysis: What Makes Tasks Hard?



KEY FINDING

PR count predicts difficulty

Hardest instances average 14.84 PRs vs 1.67 for easiest — a 9× difference

BENCHMARK STATUS

64% unsolved by everyone

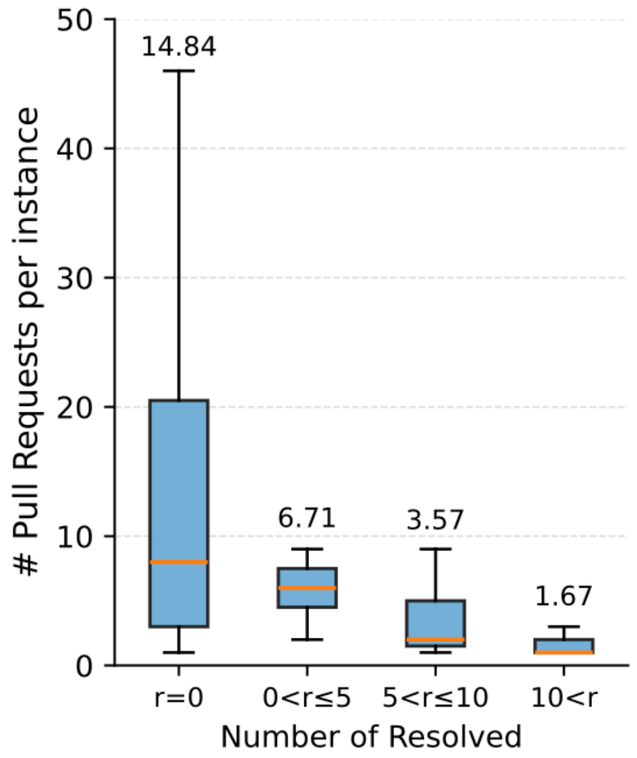
The benchmark is far from solved — genuine headroom remains for future models

MODEL BEHAVIOR

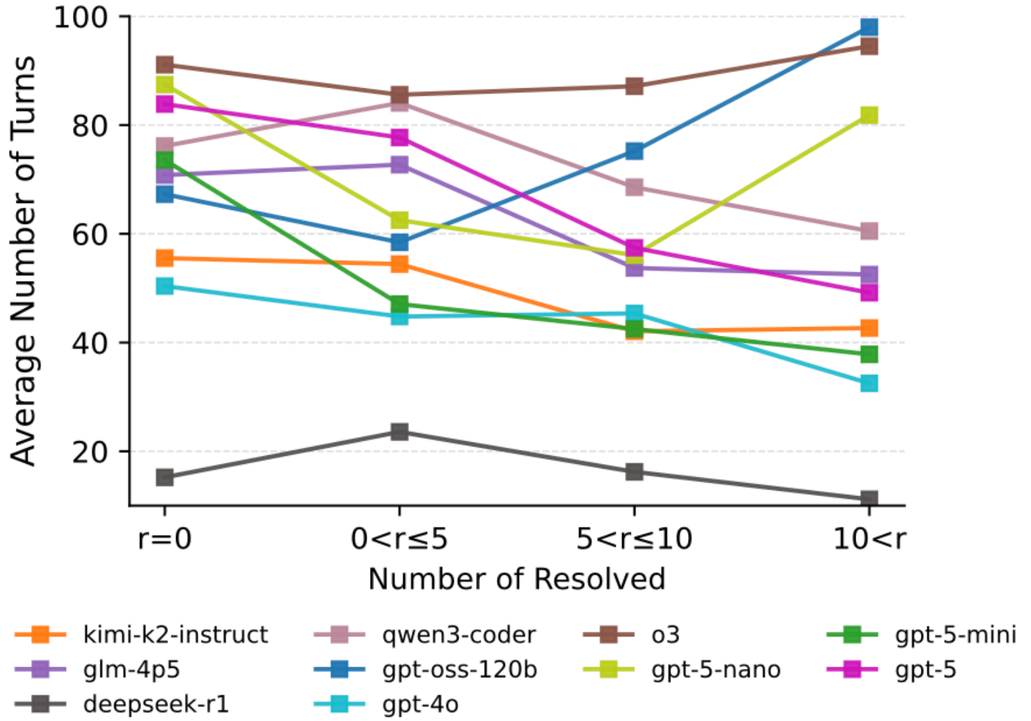
Strong models adapt, weak ones don't

Stronger models allocate more turns on harder instances — weaker models use the same turns regardless

Difficulty Analysis: What Makes Tasks Hard?



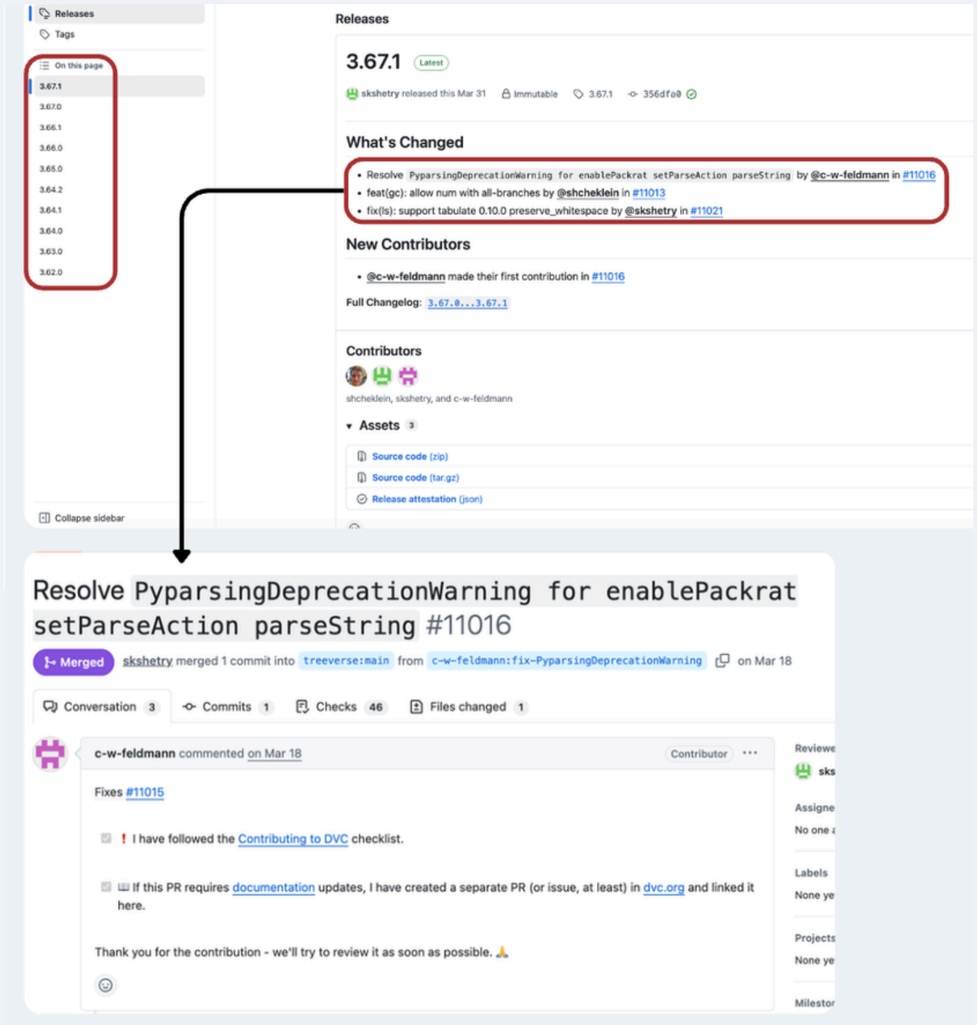
(a) Average number of pull requests per difficulty group.



(b) Average number of turns by model across difficulty groups.

Figure 6 | Difficulty analysis of SWE-EVO instances: (a) instances associated with more pull requests are harder; (b) stronger models scale turn usage with difficulty, while others show limited adaptivity.

Does providing additional context improve agent performance?



RESOLVED RATE (%) BY CONTEXT CONDITION · SWE-AGENT SCAFFOLD

With PR/issue context (main results) | Release-note only (appendix)

MODEL	NOTE ONLY	+ PR/ISSUE	CHANGE
kimi-k2-instruct	12.50%	**18.75%**	+6.3 pp
gpt-oss-120b	0.00%	**6.25%**	+6.3 pp
glm-4p5	12.50%	**16.67%**	+4.2 pp
gpt-5-08-07	16.67%	**20.83%**	+4.2 pp
gpt-4.1	8.33%	**10.42%**	+2.1 pp
deepseek-r1	6.25%	**8.33%**	+2.1 pp
o3	**6.25%**	**6.25%**	no change
gpt-5-mini	**10.42%**	**10.42%**	no change

Conclusion: Context helps most models, but modestly.

E

Limitations & Critical Analysis

Beyond what the authors acknowledge

Critical Analysis

Authors' Acknowledged Limitations

Python-Only

Multilingual and downstream-application evolution excluded

48 Instances / dvc-Heavy

54% from one repo. Each instance = 2.08pp change. Limited statistical power

Release-Note Specs Only

Misses security advisories, dependency updates, design-doc-driven refactors

Equal Test Weights

Fix Rate doesn't reflect real-world feature importance

No Code Quality Measure

Maintainability, readability, patch minimality not evaluated

Critical Perspective

Internet Access Blocked

Real engineers use search/docs. Blocking may systematically underestimate true capability

LLM-as-Judge Validity

Failure categories from gpt-5-mini, without human agreement validation. taxonomy incomplete?

Scaffold = Confound

A 29% gap for the same model (glm-5: 8% vs 37%) depending only on scaffold choice -> results reflect model + scaffold together, not model capability alone.

Curation Opacity

No explanation why only these 7 repos ended up in SWE-EVO, nor an exact count of candidates before human curation. The phrase "a few hundred" is the only information provided, leaving the selection process partially opaque.

F

Future Research Directions

Two extensions from the paper + two novel proposals

Future Research Directions

Multilingual & Polyglot Evolution

From paper

RQ: Can agents evolve codebases mixing Python, JS, Go, Rust?

Extend construction pipeline to multi-language GitHub projects. Evaluate cross-file-type dependency reasoning across language boundaries

PR-by-PR Sequential Scaffolding

From paper

RQ: Does incremental PR-level supervision bridge the SWE-Bench → SWE-EVO gap?

Create sequential PR-decomposed tasks from SWE-EVO. Compare holistic release vs incremental PR-by-PR agent performance

Context Engineering for Evolution

NOVEL DIRECTION

RQ: Can structured context (dependency graphs, change impact maps) overcome instruction-following failures?

Extract call graphs and change impact maps. Inject into agent context alongside release notes. Test whether the >60% instruction-following failure rate decreases

Evolution-Aware RL Fine-Tuning

NOVEL DIRECTION

RQ: Can RL training on SWE-EVO's Fix Rate reward signal teach models multi-file coordination?

Use Fix Rate as a reward signal for RL. Compare models trained on SWE-EVO vs SWE-Bench. Evaluate whether evolution-trained models shift failure mode distributions.

Conclusion

1. SWE-EVO fills a critical gap: benchmarks for realistic software evolution, not isolated issue repair
2. Dramatic capability gap: best model at 25% - SWE-Bench scores (up to 72.8%) don't transfer
3. Fix Rate is a valuable contribution - captures partial progress hidden by binary Resolved Rate
4. Failure modes differ by model capability: strong models misread specs; weak models break on tools
5. Novel proposals: context engineering + evolution-aware RL to directly address root failure causes

github.com/SWE-EVO/SWE-EVO | https://huggingface.co/datasets/Fsoft-AIC/SWE-EVO | arXiv:2512.18470